

Universidad Mariano Galvez de Guatemala.
Facultad de Ingeniería en Sistemas de Información y Ciencias de la
Computación.
Curso: 021- Métodos Numéricos
Ing. MA. Carmelo Estuardo Mayén Monterroso



INFORME FINAL

Plataforma de Ejecución, Trazabilidad y Comparación de Métodos Numéricos con Arquitectura Híbrida

Nombres:

César Gustavo Castillo García

Luis Enrique Escobar Vides

Eduardo Leonel Monzón López

Bayrón Esau Morales Mazariegos

José Adrián Monterroso Melgar

Nathalie María Amalia Carbajal García

Carné:

1790-24-20456

1790-24-26001

1790-24-8668

1790-24-26929

1790-24-14368

1790-24-15648

Guatemala, 27 de mayo de 2026

INDICE

INTRODUCCIÓN	3
1. RESUMEN EJECUTIVO DEL PROYECTO DESARROLLADO	4
1. INFORMACIÓN GENERAL DEL PROYECTO	4
2. SOLUCIÓN DESARROLLADA.....	4
3. FLUJO ASÍNCRONO Y TRAZABILIDAD.....	5
4. BENEFICIOS Y RESULTADOS OBTENIDOS	5
2. DESCRIPCIÓN DE LOS MÉTODOS NUMÉRICOS IMPLEMENTADOS Y SU CORRECTITUD MATEMÁTICA.....	6
2.1 MÉTODOS DE BÚSQUEDA DE RAÍCES DE ECUACIONES NO LINEALES	6
2.1.1 MÉTODO DE NEWTON-RAPHSON	6
2.1.2 MÉTODO DE LA SECANTE	7
2.1.3 MÉTODO DE MÜLLER	8
2.2 MÉTODOS DE RESOLUCIÓN DE SISTEMAS LINEALES.....	9
2.2.1 MÉTODO DE GAUSS.....	9
2.2.2 MÉTODO DE GAUSS-SEIDEL	10
2.2.3 MÉTODO DE GAUSS-JORDÁN.....	11
3. REFLEXIÓN SOBRE LOS RETOS TÉCNICOS ENCONTRADOS Y CÓMO SE RESOLVIERON	12
3.1 PROBLEMAS DE CONVERGENCIA Y VALIDACIÓN MATEMÁTICA	12
3.2 PROBLEMAS DE ESTABILIDAD Y RECURSOS	13
3.3 PROBLEMAS DE VISUALIZACIÓN Y EXPERIENCIA DE USUARIO	13
4. CONCLUSIONES	14
5. RECOMENDACIONES.....	15
6. DISTRIBUCIÓN DEL TRABAJO POR INTEGRANTE DEL EQUIPO.....	16
7. ANEXOS	17
7.1 ENLACE DEL REPOSITORIO DE GITHUB	17
7.2 ENLACE DE LA CARPETA DE GOOGLE DRIVE	17
7.3 ENLACE VIDEO DEMOSTRACIÓN	17
7.3 Enlace Video Demostración.....	17

INTRODUCCIÓN

En el ámbito de la ingeniería y las ciencias aplicadas, los métodos numéricos constituyen una herramienta fundamental para la resolución de problemas matemáticos que carecen de solución analítica exacta o cuya resolución simbólica resulta impracticable. Desde la búsqueda de raíces de ecuaciones no lineales hasta la resolución de grandes sistemas de ecuaciones lineales, algoritmos como Newton-Raphson, eliminación gaussiana o Gauss-Seidel han sido empleados durante siglos. Sin embargo, la forma tradicional de enseñar y ejecutar estos métodos presenta limitaciones significativas: los entornos de escritorio son síncronos, no ofrecen trazabilidad iteración por iteración y dificultan la comparación objetiva entre diferentes algoritmos.

El presente proyecto, titulado “APLI METODOS”, nace con el objetivo de superar estas limitaciones mediante una aplicación web moderna, asíncrona y completamente contenerizada. La plataforma permite a los estudiantes y profesionales de la ingeniería configurar y ejecutar seis métodos numéricos clásicos (Newton-Raphson, Secante, Müller, Gauss, Gauss-Seidel y Gauss-Jordan) de forma remota, sin bloqueos de interfaz, gracias a una arquitectura basada en colas de mensajes con Redis. Cada iteración de cada método se almacena automáticamente en una base de datos SQL Server, lo que proporciona una trazabilidad completa y posibilita la generación de gráficas de convergencia (error vs. iteración) en tiempo real. Además, la plataforma incluye un módulo de comparación que superpone las curvas de distintos métodos sobre un mismo problema, facilitando el análisis didáctico de la velocidad y estabilidad de cada algoritmo.

Tecnológicamente, el sistema se compone de cinco servicios independientes ejecutándose en contenedores Docker: un frontend desarrollado con React y TypeScript, una API principal escrita en .NET 10 que orquesta los trabajos, un worker numérico en Python 3.12 que realiza los cálculos pesados utilizando SymPy y NumPy, una cola Redis para el desacoplamiento asíncrono y una base de datos SQL Server para la persistencia de jobs e iteraciones. Todo el entorno se levanta con un único comando (`docker compose up --build`), garantizando la reproducibilidad en cualquier sistema operativo.

El presente documento constituye el Informe Final del proyecto. En él se describen los antecedentes tecnológicos, los métodos numéricos implementados, la arquitectura híbrida adoptada, los retos técnicos superados, las conclusiones obtenidas y las recomendaciones para trabajos futuros. Acompañan a este informe el Manual Técnico, el Manual de Usuario y una video demostración, todos ellos disponibles en el repositorio GitHub del proyecto.

1. Resumen ejecutivo del proyecto desarrollado

1. Información General del Proyecto

En el ámbito de la enseñanza y aplicación de los métodos numéricos, los estudiantes e ingenieros enfrentan limitaciones importantes con las herramientas tradicionales (calculadoras, hojas de cálculo, scripts locales). Estas herramientas ejecutan los algoritmos de forma síncrona, obligando al usuario a esperar sin interrupción hasta que el cálculo finaliza, lo que resulta en pantallas congeladas y tiempos muertos improductivos. Además, suelen mostrar únicamente el resultado final, ocultando el proceso iterativo de convergencia. El estudiante no puede observar cómo evoluciona el error paso a paso, lo que impide comprender la velocidad de convergencia, las oscilaciones o las causas de un fallo. Por otro lado, comparar distintos métodos (por ejemplo, Newton-Raphson vs. Secante) requiere ejecutarlos por separado, anotar manualmente los resultados y construir gráficas de forma aislada, un proceso tedioso y propenso a errores. Finalmente, la falta de reproducibilidad en entornos educativos (cada estudiante tiene un sistema operativo y versiones de software distintas) consume tiempo en configuraciones en lugar de dedicarse al análisis numérico.

2. Solución desarrollada

Para superar estas deficiencias, se diseñó e implementó APLIMETODOS, una plataforma web distribuida de código abierto cuya misión es proporcionar ejecución asíncrona, trazabilidad completa y comparación visual de seis métodos numéricos clásicos: Newton-Raphson, Secante, Müller (búsqueda de raíces), Gauss, Gauss-Seidel y Gauss-Jordan (sistemas lineales).

La plataforma se basa en una arquitectura híbrida de microservicios desacoplados, ejecutándose dentro de contenedores Docker. Está compuesta por cinco servicios:

- Frontend (React 19 + TypeScript): Interfaz de usuario donde el estudiante ingresa los parámetros (función, valores iniciales, tolerancia, etc.) y visualiza los resultados, incluyendo gráficas de convergencia y tablas de iteraciones.
- API principal (.NET 10): Orquesta los trabajos: valida los parámetros, persiste los jobs en SQL Server con estado PENDING, y los encola en Redis mediante RPUSH.
- Worker numérico (Python 3.12 + SymPy + NumPy): Se mantiene a la espera de nuevos jobs mediante BLPOP sobre la cola Redis. Cuando recibe un jobId, ejecuta el método numérico correspondiente, guardando cada iteración (valor aproximado x_i y error) en la tabla JobIterations. Al finalizar, actualiza el estado a DONE (o FAILED) y registra el tiempo de ejecución.

- Cola Redis 7: Actúa como mecanismo de desacoplamiento real entre la API y el worker, garantizando la asincronía.

Base de datos SQL Server 2022: Persiste toda la información de los jobs y el historial completo de iteraciones, permitiendo consultas posteriores para comparación.

3. Flujo asíncrono y trazabilidad

El flujo de trabajo es completamente no bloqueante:

1. El usuario crea un job desde el frontend → la API responde inmediatamente con el jobId y el estado PENDING.
2. La API guarda el job en SQL Server y lo encola en Redis.
3. El worker consume el jobId, actualiza el estado a RUNNING y comienza la ejecución numérica.
4. En cada iteración, el worker inserta una fila en JobIterations (iteración, xi, error, datos adicionales).
5. Al terminar, se actualiza el estado a DONE o FAILED.
6. El frontend realiza polling automático cada 5 segundos a GET /api/jobs/{id}. Cuando detecta el cambio de estado, solicita el historial de iteraciones y renderiza una gráfica de convergencia (error vs. iteración) y una tabla detallada.

4. Beneficios y resultados obtenidos

- **Asincronía real:** El usuario nunca queda bloqueado; puede crear múltiples jobs y regresar más tarde a ver los resultados.
- **Trazabilidad completa:** Cada paso de la convergencia queda registrado, lo que permite auditar, depurar y comprender el comportamiento de cada método.
- **Comparación facilitada:** Al persistir todos los jobs, la plataforma puede recuperar y superponer curvas de distintos métodos sobre un mismo problema (funcionalidad opcional implementada parcialmente).
- **Reproducibilidad:** Todo el sistema se levanta con un solo comando (docker compose up --build) en cualquier sistema operativo que tenga Docker, sin necesidad de instalar .NET, Python, Node, SQL Server o Redis.
- **Robustez:** Durante las pruebas de estrés se detectaron y corrigieron problemas críticos (falsa convergencia en funciones discontinuas, jobs colgados por spam, parser anidado, ausencia de timeout, colapso parcial del sistema). La versión final maneja adecuadamente casos de error como división por cero, matrices singulares y entradas inválidas.

2. Descripción de los métodos numéricos implementados y su correctitud matemática

2.1 Métodos de búsqueda de raíces de ecuaciones no lineales

Los métodos de esta sección aproximan la solución de una ecuación de la forma $f(x) = 0$, partiendo de uno o varios valores iniciales. La plataforma acepta funciones escritas en notación de Python (ej. $x^{**2} - 4$, $\exp(x) - 3*x$, $\sin(x) - x/2$). El usuario puede definir la función, los valores iniciales, la tolerancia y el número máximo de iteraciones.

2.1.1 Método de Newton-Raphson

Los métodos de esta sección aproximan la solución de una ecuación de la forma $f(x) = 0$, partiendo de uno o varios valores iniciales. La plataforma acepta funciones escritas en notación de Python (ej. $x^{**2} - 4$, $\exp(x) - 3*x$, $\sin(x) - x/2$). El usuario puede definir la función, los valores iniciales, la tolerancia y el número máximo de iteraciones.

Fundamento matemático

El método de Newton-Raphson es un algoritmo iterativo de convergencia cuadrática que aproxima la raíz utilizando la recta tangente a la función en cada punto. La fórmula de recurrencia es:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}, f'(x_n) \neq 0$$

La derivada puede proporcionarse explícitamente o, en este proyecto, calcularse automáticamente mediante derivación simbólica con SymPy.

Implementación en APLIMETODOS

- El worker recibe la función como cadena de texto (ej. " $x^{**2} - 2$ ").
- Convierte la cadena a una expresión de SymPy y, si el usuario no proporciona la derivada, la calcula simbólicamente.
- Evalúa $f(x_n)$ y $f'(x_n)$ numéricamente en cada iteración.
- Criterios de parada:
- $|f(x_n)| < \text{tolerancia}$ (la función se considera cero).
- $|x_{n+1} - x_n| < \text{tolerancia}$ (cambio muy pequeño).
- Se alcanza el máximo de iteraciones permitidas.

Manejo de errores:

- Si $|f'(x_n)| < 10^{-12}$, se detiene la ejecución y el job se marca como FAILED con el mensaje "Derivada cercana a cero – posible punto crítico o discontinuidad".
- Si en algún punto $|f(x_n)|$ supera un umbral (10^9), se considera que la función diverge y se finaliza con FAILED.

Verificación de correctitud

- Caso 1: $f(x) = x^2 - 2$ (raíces $\pm\sqrt{2} \approx \pm 1.41421356$), $x_0 = 1.0$, tolerancia 10^{-6} .
Resultado: Converge a **1.4142135623746899** en 4 iteraciones. Error relativo $< 10^{-12}$.
- Caso 2: $f(x) = e^x - 3x$, $x_0 = 1.0$, tolerancia 10^{-6} .
Resultado: Raíz ≈ 0.619061 , convergencia en 5 iteraciones.
- Caso de error: $f(x) = x^2$, $x_0 = 0$. Derivada nula en la primera iteración. El worker detecta la anulación y marca el job como FAILED, evitando una división por cero.

2.1.2 Método de la Secante

Fundamento

matemático

El método de la secante es una variante del método de Newton que no requiere el cálculo de la derivada. Aproxima la derivada mediante una diferencia finita entre dos iteraciones previas. La fórmula es:

$$x_{n+1} = x_n - f(x_n) \frac{x_n - x_{n-1}}{f(x_n) - f(x_{n-1})}$$

Necesita dos valores iniciales x_0 y x_1 . Su convergencia es superlineal, con orden aproximado $\phi \approx 1.618$.

Implementación en APLIMETODOS

- El usuario proporciona x_0 y x_1 .
- En cada iteración se calcula el denominador $f(x_n) - f(x_{n-1})$.
- Si el denominador es cero (o menor que 10^{-12} en valor absoluto), se lanza un error y el job finaliza como FAILED.
- Se almacenan las dos iteraciones previas para calcular el siguiente valor.

Verificación de correctitud

- **Caso 1:** $f(x) = x^3 - x - 2$, $x_0 = 1.0$, $x_1 = 2.0$, tolerancia 10^{-6} .
Resultado: Raíz ≈ 1.5213797 en 7 iteraciones. Se comprueba que el error decrece monótonamente.
- **Caso 2:** $f(x) = \sin(x)$, $x_0 = 2.0$, $x_1 = 3.0$ (raíz en $\pi \approx 3.14159265$).
Resultado: Converge a 3.1415926536 en 9 iteraciones.
- **Caso de error:** $f(x) = x^2 - 4$, $x_0 = 2$, $x_1 = -2$. En la primera iteración, $f(2) = 0$ y $f(-2) = 0$, denominador cero. El worker detecta la condición y marca FAILED.

2.1.3 Método de Müller

Fundamento

matemático

El método de Müller es una extensión de la secante que utiliza una parábola (polinomio cuadrático) que pasa por tres puntos $(x_{i-2}, f(x_{i-2}))$, $(x_{i-1}, f(x_{i-1}))$, $(x_i, f(x_i))$. Encuentra la raíz más cercana de la parábola y la toma como siguiente aproximación. A diferencia de los métodos anteriores, puede converger a raíces complejas.

Implementación en APLIMETODOS

- Se requieren tres valores iniciales x_0, x_1, x_2 .
- Se construyen las diferencias divididas y se resuelve la ecuación cuadrática.
- Se elige la raíz que minimiza la distancia al punto más reciente.
- Manejo especial de la discriminante: si es negativo, el resultado se expresa como número complejo y se registra en Resultado (pero el job continúa, ya que es una raíz válida).
- Si todas las raíces de la parábola son muy lejanas o el método no mejora, se detiene por máximo de iteraciones.

Verificación de correctitud

- **Caso real:** $f(x) = e^x - x$, $x_0 = 0$, $x_1 = 1$, $x_2 = 2$.
Resultado: Raíz real 0.5671432904097838 en 5 iteraciones.

- **Caso complejo:** $f(x) = x^2 + 1$.
Resultado: Raíces $\pm i$. El worker retorna "0+1i" y "0-1i" (según elección). Se verifica que el discriminante negativo se maneja sin fallos y el job termina con DONE.
- **Caso de discontinuidad:** $f(x) = \tan(x)$ cerca de $\pi/2$. El método puede experimentar grandes saltos; el worker lo detecta y finaliza con FAILED por "posible discontinuidad".

2.2 Métodos de resolución de sistemas lineales

2.2.1 Método de Gauss

Fundamento

matemático

El método de Gauss transforma la matriz aumentada $[A \mid \mathbf{b}]$ en una matriz triangular superior mediante operaciones elementales de fila (intercambios, sumas de múltiplos). Luego resuelve por sustitución regresiva. El pivoteo parcial consiste en seleccionar el elemento de mayor valor absoluto en la columna actual como pivote para mejorar la estabilidad numérica.

Implementación en APLIMETODOS

- La matriz se ingresa como cadena de texto con filas separadas por ; y columnas por , (ej. "3,2;2,6").
- Se convierte a un array de NumPy de tipo float.
- Para cada columna k desde 0 hasta n-1:
 - Se busca el índice max_row de la fila con el mayor valor absoluto en la columna k (desde k hasta n-1).
 - Se intercambian las filas k y max_row.
 - Si el pivote es cero, se declara matriz singular y el job falla.
 - Se eliminan los elementos debajo del pivote restando múltiplos de la fila k.
- Sustitución regresiva para obtener la solución.

Verificación de correctitud

- **Caso 1:** Sistema
$$\begin{cases} 3x + 2y = 5 \\ 2x + 6y = 8 \end{cases}$$

Resultado: $x = 1, y = 1$ (solución exacta).

- **Caso 2:** Sistema mal condicionado $\begin{cases} 1e - 10x + y = 1 \\ x + y = 2 \end{cases}$

Resultado: Aproximadamente $x \approx 1, y \approx 1$. El pivoteo parcial evita la división por números muy pequeños.

- **Caso de error:** Matriz singular $\begin{cases} 1x + 2y = 3 \\ 2x + 4y = 6 \end{cases}$ (determinante cero).

Resultado: El worker detecta pivote nulo y marca FAILED con el mensaje "Matriz singular, no se puede resolver".

2.2.2 Método de Gauss-Seidel

Fundamento

matemático

Es un método iterativo para sistemas lineales. En cada paso, actualiza cada componente de x usando los valores más recientes disponibles. La fórmula para la componente i -ésima en la iteración $k + 1$ es:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij} x_j^{(k+1)} - \sum_{j=i+1}^n a_{ij} x_j^{(k)} \right)$$

Converge si la matriz es diagonalmente dominante o definida positiva.

Implementación en APLIMETODOS

- El usuario ingresa la matriz, el vector b , una aproximación inicial x_0 (opcional, por defecto vector de ceros), la tolerancia y el número máximo de iteraciones.
- En cada iteración se actualizan los valores de x secuencialmente (sobrescribiendo el vector).
- Se calcula el error relativo como la norma infinita de la diferencia entre iteraciones sucesivas.
- Si la matriz no es diagonalmente dominante, puede no converger; el algoritmo se detiene por máxima iteración y marca Converged = false.

Verificación de correctitud

- **Caso convergente:** Matriz diagonalmente dominante $A = \begin{bmatrix} 4 & 1 \\ 1 & 3 \end{bmatrix}, b = [5, 4]^T$. Solución exacta $x = 1, y = 1$.
Resultado: Converge en 6 iteraciones con error $< 10^{-6}$.

- **Caso no convergente:** Matriz $A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$, $b = [5, 11]^T$. No es diagonalmente dominante.
Resultado: El algoritmo no converge después de 1000 iteraciones; se detiene y el job se marca como DONE pero Converged = false. El usuario recibe una advertencia.

2.2.3 Método de Gauss-Jordán

Fundamento

matemático

El método de Gauss-Jordan extiende la eliminación de Gauss hasta reducir la matriz aumentada a su forma escalonada reducida (RREF). En esta forma, la solución del sistema se lee directamente en la última columna. Requiere más operaciones que Gauss, pero proporciona la solución sin necesidad de sustitución regresiva.

Implementación en APLIMETODOS

- Se construye la matriz aumentada $[A \mid b]$.
- Para cada columna k , se selecciona el pivote máximo (pivoteo parcial), se normaliza la fila del pivote dividiéndola por el pivote, y luego se eliminan los elementos por encima y por debajo del pivote en todas las filas.
- Si el pivote es cero en alguna etapa, se declara matriz singular.
- La solución es la última columna después de procesar todas las columnas.

Verificación de correctitud

- **Caso estándar:** Mismo sistema $3x + 2y = 5, 2x + 6y = 8$.
Resultado: La matriz aumentada se transforma a $\left[\begin{array}{cc|c} 1 & 0 & 1 \\ 0 & 1 & 1 \end{array} \right]$. Lectura directa de la solución $x = 1, y = 1$.
- **Caso de error:** Matriz singular $A = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$. En la primera columna, después de buscar pivote, el elemento (2,1) también es cero. Se detecta singularidad y el job termina en FAILED con mensaje adecuado.

3. Reflexión sobre los retos técnicos encontrados y cómo se resolvieron

Durante el ciclo de desarrollo y pruebas de **APLIMETODOS**, se identificaron múltiples problemas que abarcan desde errores matemáticos (falsa convergencia, mal manejo de funciones discontinuas) hasta fallos de infraestructura (jobs colgados, parser anidado, ausencia de timeout). A continuación, se describen los hallazgos más críticos según los reportes de QA y las acciones correctivas implementadas

3.1 Problemas de convergencia y validación matemática

Problema	Método(s) afectado(s)	Solución implementada
 Convergencia falsa en funciones sin raíz real (ej. x^2+4)	• Secante	Se añadió una verificación posterior: después de que el método reporta convergencia, se evalúa $ f(x^*) $. Si no es cercano a cero (tolerancia), se fuerza el estado FAILED con mensaje "Raíz falsa detectada" .
 Falsa convergencia cerca de discontinuidades (ej. $\tan(x)$ en $\pi/2$)	• Secante • Newton-Raphson	Se incorporó detección de valores extremos ($ f(x) > 1e9$) y de cambios abruptos entre iteraciones. Si se detecta una posible asíntota, el job finaliza con FAILED .
 No convergencia en funciones que sí convergen (ej. $e^{-1-x} - x$ con Müller)	• Müller	Se corrigió la actualización de los tres puntos en cada iteración y se mejoró el criterio de parada (ahora se basa en el error relativo y no solo en la diferencia de x).
 Convergencia extremadamente lenta o ciclos (ej. $(x-1)^5$ con Secante)	• Secante • Newton-Raphson	Se implementó un contador de iteraciones sin mejora significativa y se agregó un <i>timeout</i> global de 15 segundos por job. Si se detecta estancamiento, el job se detiene con estado FAILED .

3.2 Problemas de estabilidad y recursos

Problema	Síntoma	Solución implementada
 Jobs colgados en estado PENDING	Al enviar muchos jobs rápidamente, el sistema se congelaba.	 Se corrigió el bucle de consumo de Redis <code>BLPOP</code> con <code>timeout</code> y se agregó idempotencia: el worker verifica el estado en BD antes de ejecutar.
 Parser colgado por expresiones anidadas PENDING	<code>((((((<>))))))</code> dejaba el job en PENDING.	 Se validó la función en el worker: se rechazan expresiones con paréntesis sin operadores y con profundidad > 10.
 Ausencia de timeout	Jobs con funciones muy complejas se ejecutaban indefinidamente.	 Se implementó un <code>timeout</code> de 15 segundos por job usando <code>signal.alarm</code> (Unix) y un contador de tiempo en el bucle principal.
 Colapso parcial del sistema	Bajo estrés, la API devolvía 500 y el worker se reiniciaba.	 Se añadió un middleware global de excepciones en la API, se limitaron las inserciones masivas y se configuraron reintentos en el frontend.

3.3 Problemas de visualización y experiencia de usuario

Problema	Lugar	Solución implementada
 Tabla de Newton-Raphson no mostraba $f(x_i)$, $f'(x_i)$ ni error	Frontend	Se modificó el DTO de iteraciones para incluir explícitamente estos campos y se actualizó la tabla en React.
 Tabla de Müller mostraba columnas vacías	Frontend	Se corrigió el mapeo de los datos adicionales ($x_0, x_1, x_2, f(x_2)$) que llegan en el campo <code>DatosAdicionales</code> .
 Gráfica incompleta en jobs fallidos	Frontend	Se ajustó el polling para que, incluso si el estado es FAILED , se carguen las iteraciones acumuladas y se dibuje la gráfica parcial.
 Mensajes de error poco precisos (ej. "función inválida" aparece como "discontinuidad")	Frontend	Se enriquecieron los mensajes de error en el worker, distinguiendo entre <code>NameError</code> (variable no definida), <code>ValueError</code> (dominio) y <code>ZeroDivisionError</code> .

4. Conclusiones

1. **La plataforma demuestra estabilidad operativa bajo condiciones nominales**, sin caídas del worker o de los contenedores Docker durante las pruebas funcionales estándar. Sin embargo, se identificaron vulnerabilidades críticas en el manejo de funciones sin raíces reales y discontinuidades, evidenciando una dependencia excesiva del criterio de diferencia entre iteraciones sin validación posterior de la imagen funcional.
2. **El método de la Secante presenta un comportamiento errático en escenarios de convergencia falsa**, reportando soluciones inexistentes para ecuaciones como $x^2 + 4 = 0$. Este fenómeno obedece a la ausencia de una verificación complementaria que evalúe la magnitud de $f(x^*)$ una vez que la diferencia entre iteraciones cae por debajo de la tolerancia.
3. **La arquitectura actual carece de validaciones de entrada robustas** a nivel de la API y el frontend, permitiendo el ingreso de parámetros extremos (iteraciones máximas del orden de 10^9 , tolerancias negativas, funciones con anidamiento excesivo de paréntesis) que comprometen la integridad del procesamiento y pueden inducir condiciones de denegación de servicio.
4. **La trazabilidad visual presenta deficiencias en la representación tabular**, específicamente en los métodos de Newton-Raphson (ausencia de columnas para $f(x_i)$, $f'(x_i)$ y error relativo) y Müller (columnas vacías para x_0 , x_1 , x_2 y $f(x_2)$), lo que limita el valor pedagógico y analítico de la plataforma.
5. **No se implementó un mecanismo de timeout global para la ejecución de jobs**, lo que permite que procesos mal condicionados o no convergentes consuman recursos indefinidamente, afectando la disponibilidad del worker y la capacidad de respuesta del sistema ante cargas sostenidas.
6. **El módulo de comparación de métodos carece de retroalimentación granular sobre las causas de fallo**, ofreciendo únicamente un estado binario de éxito o fracaso sin desglose de iteraciones parciales ni mensajes contextuales, lo que dificulta el diagnóstico y la mejora algorítmica.

5. Recomendaciones

1. **Implementar un doble criterio de convergencia** que combine la diferencia relativa entre iteraciones con la evaluación de $|f(x^*)|$ por debajo de la tolerancia. En caso de discrepancia, el job debe marcarse como FAILED con el mensaje "Convergencia aparente no respaldada por el valor funcional".
2. **Establecer validaciones paramétricas estrictas en la capa de API**, rechazando solicitudes que superen los siguientes umbrales: iteraciones máximas > 1000 , tolerancia fuera del rango $[10^{-15}, 0.1]$, longitud de función > 120 caracteres, profundidad de paréntesis anidados > 10 , y matrices con celdas no numéricas o vacías.
3. **Incorporar un timeout configurable por job** (valor sugerido: 30 segundos) utilizando un mecanismo de interrupción asíncrona (por ejemplo, `signal.alarm` en Unix o un temporizador con hilo de control). Al excederse, el estado debe actualizarse a FAILED con el mensaje "Tiempo de ejecución excedido".
4. **Rediseñar los DTO de iteraciones** para los métodos de Newton-Raphson y Müller, asegurando que el worker transmita todos los campos relevantes (`xi`, `f_xi`, `f_prima_xi`, `xi_mas_1`, `error_aproximado` en el primer caso; `x0`, `x1`, `x2`, `f_x2` en el segundo) y que el frontend los renderice de forma consistente.
5. **Enriquecer la semántica de los mensajes de error** mediante la captura específica de excepciones: `NameError` $\rightarrow$ "Función inválida: variable no reconocida", `ValueError` $\rightarrow$ "Operación fuera del dominio matemático", `ZeroDivisionError` $\rightarrow$ "División por cero en la fórmula iterativa", `OverflowError` $\rightarrow$ "Desbordamiento numérico".
6. **Modificar el flujo de visualización de iteraciones** para que, incluso en jobs con estado FAILED, el frontend solicite y represente las iteraciones acumuladas hasta el momento del fallo, generando la gráfica de convergencia parcial y facilitando el análisis de la causa raíz.

6. Distribución del trabajo por integrante del equipo

Carnet	Nombre	Rol principal	Contribuciones específicas
1790-24-14368	José Adrián Monterroso Melgar	Frontend / Base de datos	Desarrollo de componentes frontend en React, integración de gráficas y polling. Diseño e implementación del esquema de base de datos (tablas <code>Jobs</code> , <code>JobIterations</code> , índicea).
1790-24-26001	Luis Enrique Escobar Vides	Diseñador UX / Manual de Usuario	Diseño de experiencia de usuario (UX) de la interfaz, wireframes, flujos de navegación. Elaboración completa del Manual de Usuario (guía paso a paso, capturas, interpretación de resultados).
1790-24-26929	Bayrón Esau Morales Mazariegos	Backend / Worker numérico	Implementación de los seis métodos numéricos en Python (Newton-Raphson, Secante, Müller, Gauss, Gauss-Seidel, Gauss-Jordan). Lógica de consumo de Redis (<code>ELPO</code> y guardado de iteraciones en la base de datos.
1790-24-20456	César Gustavo Castillo García	QA / Control de calidad	Ejecución de pruebas funcionales y de estrés, identificación y documentación de bugs, verificación de correcciones, control de calidad.
1790-24-8668	Eduardo Leonel Monzón López	QA / Control de calidad	Ejecución de pruebas funcionales y de estrés, identificación y documentación de bugs, verificación de correcciones, control de calidad.
1790-24-15648	Nathalie Maria Amaia Carbajal Garcia	Scrum Master / Documentación técnica	Gestión del equipo bajo metodología ágil (Scrum). Redacción del Manual Técnico (arquitectura, diagramas C4, modelo BD, variables de entorno, pruebas). Elaboración de diagramas y actas.

7. Anexos

7.1 Enlace del Repositorio de GitHub

<https://github.com/JoseAdrianMelgar/APLIMETODOS>

7.2 Enlace de la carpeta de Google Drive

https://drive.google.com/drive/folders/1cnYDL53ukhq_1X-WcVCjIABD-lkrbvgZ?usp=sharing

7.3 Enlace Video Demostración

<https://drive.google.com/file/d/1ohtznbpoWFvYZ7QZdhLXs-d7X1LhWYpk/view?usp=sharing>